Cours – Automatisation du Développement

Commandes Maker & Intégration Continue avec GitHub Actions

Objectifs du cours

Ce cours vise à comprendre deux aspects essentiels de l'automatisation dans le développement moderne :

- **Les commandes Maker**, qui facilitent la génération de code et accélèrent le prototypage.
- **L'intégration continue (CI)**, illustrée à travers **GitHub Actions**, qui permet d'automatiser les tests et l'analyse d'un projet à chaque modification.

Partie 1 — Les Makers

1. Introduction

Les *Makers* sont des outils intégrés à de nombreux frameworks pour **générer automatiquement du code standardisé**.
Ils permettent aux développeurs de se concentrer sur la logique métier plutôt que sur la configuration initiale.

2. Pourquoi utiliser un Maker ?

- **Gain de temps** : création instantanée de classes structurées
- **Migration** : modification de la base de données pour représenter des changements de schéma de manière simple, sans faire de SQL.
- **Uniformisation du code** : tous les développeurs utilisent les mêmes conventions.
- **Réduction des erreurs** : les fichiers générés respectent le squelette attendu par le framework.
- **Outil pédagogique** : aide à comprendre la structure d'un projet.

3. Exemples selon les frameworks

Chaque framework dispose de son propre outil de génération :

- **Symfony** : `php bin/console make:controller`, `make:entity`, `make:form`
- **Laravel** : `php artisan make:controller`, `make:model`, `make:command`
- **Angular (JS)** : `ng generate component`, `ng generate service`
- etc.

Malgré leur diversité, la logique reste la même : on décrit *ce qu'on veut créer*, l'outil se charge du reste.

4. Exemple — Symfony

Une commande courante :

```
php bin/console make:controller BlogController
```

Résultat :

- Un fichier `src/Controller/BlogController.php` contenant une méthode `index()`.
- Un fichier Twig associé (ex : `templates/blog/index.html.twig`).

Autres exemples utiles :

```
php bin/console make:entity Article
php bin/console make:form ArticleType
php bin/console make:command import:articles
php bin/console make:migration
```

5. Bonnes pratiques

- Relire le code généré pour comprendre la structure.
- Personnaliser les méthodes selon les besoins du projet.
- Ne pas « tout générer » sans raison : comprendre pourquoi on crée chaque entité ou contrôleur.
- Coupler l'usage des Makers à un système de versionnement (Git), pour suivre les changements.

Partie 2 — Intégration Continue (CI) et GitHub Actions / Gilab CI

1. Qu'est-ce que la CI/CD ?

- **CI (Continuous Integration)** : chaque fois qu'un développeur pousse du code, une plateforme exécute des tests ou analyses automatiques.
- **CD (Continuous Delivery/Deployment)** : les changements validés peuvent être déployés automatiquement sur un environnement cible.

2. Objectifs de la CI

- Maintenir un code stable à tout moment.
- Automatiser les tests, le linting, la compilation, etc.
- Déceler rapidement les régressions.
- Éliminer la dépendance à un poste local (tout tourne sur un serveur distant).

3. Services CI populaires

- **GitHub Actions** (intégré à GitHub)
- **GitLab CI** (integré à Gitlab)
- **Travis CI**
- **CircleCI**
- **Jenkins**

Tous reposent sur la même idée : définir un **pipeline automatisé** à partir d'un fichier de configuration.

4. GitHub Actions — fonctionnement global

Un workflow GitHub Actions se définit dans un fichier YAML situé dans `.github/workflows/`.
Il contient :

- **Des déclencheurs (on:)** : quand exécuter le workflow (push, pull request…)
- **Des jobs** : des groupes d'actions à exécuter
- **Des steps** : les commandes concrètes exécutées

Exemple simple : Lint JavaScript sur github actions

```yaml
name: lint
on:
  pull_request:

jobs:
  eslint:
    runs-on: ubuntu-latest

    steps:
      - uses: actions/checkout@v4
      - name: Setup Node
        continue-on-error: false
        uses: actions/setup-node@v3
        with:
          node-version: 20
      - run: npm ci
      - run: npm run lint
```

Ce que fait ce workflow :

1. Il s'exécute à chaque *pull request*.
2. Il télécharge le code (`actions/checkout`).
3. Il installe Node.js version 20.
4. Il installe les dépendances du projet.
5. Il lance le *linting* (`npm run lint`).
6. Si erreur => il stoppe la CI (`continue-on-error: false`)

4. Gitlab Ci - fonctionnement global

Un workflow Gitlab CI se définit dans un fichier YAML situé dans `.gitlab-ci.yml`.
Il contient notamment :

- **Des pré-job (before_script)** : des prérequis à executer
- **Des étapes (stages)** : des groupes d'actions à exécuter, on choisit quand
- **Des actions (jobs)** : les commandes concrètes exécutées dans chaque job

Exemple simple : Linter PHP sur gitlab CI

```yaml
image: php:8.3
stages:
```

```
    - test

  before_script:
    - cp .env.test .env
    - apt-get update && apt-get install -y unzip git curl
    - curl -sS https://getcomposer.org/installer | php -- --install-
  dir=/usr/local/bin --filename=composer
    - composer install --no-interaction --no-progress --prefer-dist --no-
  scripts

  lint-job:
    stage: lint
    allow_failure: false
    script:
      - echo "Running linter... "
      - composer run phpcs || exit 1
      - echo "linter completed."
```

Ce que fait ce workflow :

1. Il s'exécute à chaque push
2. Il télécharge le code copie le fichier .env, et installe les dépendances nécessaires
3. Il lance les tests unitaires (`composer run tests`).
4. Si erreur => il stoppe la CI (`allow_failure: false` + `composer run phpcs || exit 1`)

6. Github actions : Concepts clés

- `runs-on`: choisis l'environnement (Ubuntu, Windows, etc.).
- `steps`: séquence d'actions exécutées.
- `actions/checkout`: télécharge le code depuis GitHub.
- `uses:` indique une action préexistante.
- `run:` exécute une commande shell.
- `with:` permet de passer des paramètres.

==> il y a des équivalents sur gitlab et toutes les plateformes, à voir dans la doc.

7. Variables et secrets

GitHub Actions permet de définir des **variables d'environnement** ou des **secrets** sécurisés :

```
  env:
    NODE_ENV: test
```

Les secrets (comme des mots de passe) se configurent dans `Settings > Secrets and variables`.

==> il y a des équivalents sur gitlab et toutes les plateformes, à voir dans la doc.

À retenir

- Les **Makers** sont des assistants de génération de code : ils accélèrent la mise en place et uniformisent le projet.
- La **CI** garantit un code propre et stable à chaque contribution.
- **GitHub Actions / Gitlab CI** offre un environnement automatisé pratique et intégré directement à GitHub/Gitlab.
- L'automatisation ne remplace pas la compréhension : elle amplifie la productivité lorsqu'on maîtrise le fond technique.